

In [2]:

```
#Day 6  
#Sets and Dictionary
```

In [6]:

```
var = {1,2,3,4,5,6,7,8,9,10}  
print(type(var))  
print(var)  
for item in var:  
    print(item, end = " ")
```

```
<class 'set'>  
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}  
1 2 3 4 5 6 7 8 9 10
```

In [8]:

```
var2 = set(["Rohan","Sohan", "Joe", "Kapil"])  
print(type(var2))  
print(var2)  
for item in var2:  
    print(item, end = " ")
```

```
<class 'set'>  
{'Kapil', 'Sohan', 'Joe', 'Rohan'}  
Kapil Sohan Joe Rohan
```

In [13]:

```
var3 = set(["Rohan","Sohan", "Joe", "Kapil",123,456,754,45.66, True,False])  
print(type(var3))  
print(len(var3))  
print(var3)  
for item in var3:  
    print(item, end = " ")
```

```
<class 'set'>  
10  
{False, 'Kapil', 'Rohan', 'Sohan', True, 456, 45.66, 754, 'Joe', 123}  
False Kapil Rohan Sohan True 456 45.66 754 Joe 123
```

In [16]:

```
var4 = set(("Rohan","Sohan"), "Joe", "Kapil",123,456,754,45.66, True,False])  
print(type(var4))  
print(len(var4))  
print(var4)  
for item in var4:  
    print(item, end = " ")
```

```
<class 'set'>  
9  
{False, 'Kapil', True, 456, 45.66, ('Rohan', 'Sohan'), 754, 'Joe', 123}  
False Kapil True 456 45.66 ('Rohan', 'Sohan') 754 Joe 123
```

In [18]:

```
#var5 = {}    #this will be declare as dictionary
var5 = set({})
print(type(var5))
```

<class 'set'>

In [19]:

```
var6 = {1,2,4,4,5,8,9,9,10}
print("Length of set var6 = " , len(var6))
print("Return set with unique elements:",var6)
```

Length of set var6 = 7

Return set with unique elements: {1, 2, 4, 5, 8, 9, 10}

In [26]:

```
var7 = {1,2,4,5,8,9,10}
print(var7)
var7.add(23)
var7.add(231)
var7.add(2311)
print(var7)
var7.update([31,311,3111])
print(var7)
```

{1, 2, 4, 5, 8, 9, 10}

{1, 2, 4, 5, 231, 8, 9, 10, 2311, 23}

{1, 2, 4, 5, 231, 8, 9, 10, 2311, 3111, 23, 311, 31}

In [27]:

```
print(var7)
var7.discard(10)
var7.discard(2311)
print(var7)
```

{1, 2, 4, 5, 231, 8, 9, 10, 2311, 3111, 23, 311, 31}

{1, 2, 4, 5, 231, 8, 9, 3111, 23, 311, 31}

In [28]:

```
var7.discard(11111)
print(var7)
```

{1, 2, 4, 5, 231, 8, 9, 3111, 23, 311, 31}

In [29]:

```
var7.remove(3111)
print(var7)
```

{1, 2, 4, 5, 231, 8, 9, 23, 311, 31}

In [30]:

```
var7.remove(11111)
print(var7)
```

-
KeyError Traceback (most recent call last)

```
<ipython-input-30-2b286b814d8f> in <module>
----> 1 var7.remove(11111)
      2 print(var7)
```

KeyError: 11111

In [32]:

```
print(var7)
var7.pop()
var7.pop()
print(var7)
```

```
{1, 2, 4, 5, 231, 8, 9, 23, 311, 31}
{4, 5, 231, 8, 9, 23, 311, 31}
```

In [33]:

```
var7.pop()
print(var7)
```

```
{5, 231, 8, 9, 23, 311, 31}
```

In [49]:

```
table2 = {2,4,6,8,10,12,14,16,18,20}
table4 = {4,8,12,16,20,24,28,32,36,40}
print(type(table2))
print(type(table4))
print(len(table2))
print(len(table4))
```

```
<class 'set'>
<class 'set'>
10
10
```

In [38]:

```
table2union4 = table2 | table4
print(type(table2union4))
print(len(table2union4))
print(table2union4)
```

```
<class 'set'>
15
{2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 24, 28, 32, 36, 40}
```

In [39]:

```
table2union4 = table2.union(table4)
print(type(table2union4))
print(len(table2union4))
print(table2union4)
```

```
<class 'set'>
15
{2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 24, 28, 32, 36, 40}
```

In [40]:

```
table2intersection4 = table2 & table4
print(type(table2intersection4))
print(len(table2intersection4))
print(table2intersection4)
```

```
<class 'set'>
5
{4, 8, 12, 16, 20}
```

In [41]:

```
table2intersection4 = table2.intersection(table4)
print(type(table2intersection4))
print(len(table2intersection4))
print(table2intersection4)
```

```
<class 'set'>
5
{4, 8, 12, 16, 20}
```

In [43]:

```
table6 = {6,12,18,24,30}
print(table2)
print(table4)
print(table6)
```

```
{2, 4, 6, 8, 10, 12, 14, 16, 18, 20}
{32, 4, 36, 8, 40, 12, 16, 20, 24, 28}
{6, 12, 18, 24, 30}
```

In [50]:

```
print(table2.intersection(table4))
print(table2)
```

```
{4, 8, 12, 16, 20}
{2, 4, 6, 8, 10, 12, 14, 16, 18, 20}
```

In [51]:

```
#removes the items from the original set that are
#not present in both the sets (all the sets if more than one are specified).
table2.intersection_update(table4,table6)
print(table2)
```

```
{12}
```

In [55]:

```
var8 = {1,2,3,4,5}
var9 = {4,5,6,7,8}
print(var8 - var9)
print(var8.difference(var9))
```

```
{1, 2, 3}
{1, 2, 3}
```

In [56]:

```
print(var9 - var8)
print(var9.difference(var8))
```

```
{8, 6, 7}
{8, 6, 7}
```

In [58]:

```
print(var8 ^ var9)
print(var8.symmetric_difference(var9))
```

```
{1, 2, 3, 6, 7, 8}
{1, 2, 3, 6, 7, 8}
```

In [63]:

```
var9 = {1,2,3,4,5,6,7,8,9,10}
var10 = {2,4,6,8,10}
var11 = {4,2,6,10,8}

print(var9 > var10) #Is var9 is the superset for var10
print(var9 < var10) #Is var9 is the subset for var10
print(var9 == var10)
print(var10 == var11)
```

```
True
False
False
True
```

In [64]:

```
Frozenset = frozenset([1,2,3,4,5,6])
print(type(Frozenset))
print(len(Frozenset))
print(Frozenset)
```

```
<class 'frozenset'>
6
frozenset({1, 2, 3, 4, 5, 6})
```

In [65]:

```
Frozenset.add(45)
```

```
-----
-
AttributeError                                Traceback (most recent call las
t)
```

```
<ipython-input-65-d20a392db040> in <module>
```

```
----> 1 Frozenset.add(45)
```

AttributeError: 'frozenset' object has no attribute 'add'

In [66]:

```
Frozenset.remove(3)
```

```
-----
-
AttributeError                                Traceback (most recent call las
t)
```

```
<ipython-input-66-bce4b0f33b69> in <module>
```

```
----> 1 Frozenset.remove(3)
```

AttributeError: 'frozenset' object has no attribute 'remove'

In [72]:

```
var12 = {2,4,6,8,10}
var13 = {7,14,21,28,35}
print(var12.isdisjoint(var13))
print(var12.issuperset(var13))
print(var12.issubset(var13))
```

```
True
```

```
False
```

```
False
```

In [73]:

```
#Dictionary
```

In [76]:

```
Employee = {"Name":"Rohan", "Age":43, "Dept":"Account"}
print(type(Employee))
print(len(Employee))
print(Employee)
```

```
<class 'dict'>
```

```
3
```

```
{'Name': 'Rohan', 'Age': 43, 'Dept': 'Account'}
```

In [77]:

```
dict1 = {}
print(type(dict1))
```

```
<class 'dict'>
```

In [80]:

```
Language = dict({1:"C",2:"C++",3:"Java",4:"Python",5:"Front-end Developer (MERN)",5:"Matlab"})
print(type(Language))
print(len(Language))
print(Language)
```

```
<class 'dict'>
5
{1: 'C', 2: 'C++', 3: 'Java', 4: 'Python', 5: 'Matlab'}
```

In [87]:

```
print(Language[1])
print(Language[2])
print(Language[3])
print(Language[4])
print(Language[5])
print("=====")
for i in range(1,6):
    print(Language[i])
```

```
C
C++
Java
Python
Matlab
=====
C
C++
Java
Python
Matlab
```

In [85]:

```
print(Employee["Name"])
print(Employee["Age"])
print(Employee["Dept"])
```

```
Rohan
43
Account
```

In [89]:

```
Language[6] = input("Enter the language")
print(Language)
```

```
Enter the languageMachine Learning
{1: 'C', 2: 'C++', 3: 'Java', 4: 'Python', 5: 'Matlab', 6: 'Machine Learning'}
```

In [90]:

```
del Language[3]
print(Language)
```

```
{1: 'C', 2: 'C++', 4: 'Python', 5: 'Matlab', 6: 'Machine Learning'}
```

In [91]:

```
del Language[32]
```

-

KeyError Traceback (most recent call last)

<ipython-input-91-ae37f618c11f> in <module>

----> 1 del Language[32]

KeyError: 32

In []:

```
del Language
```

In [92]:

```
print(Language)
```

-

NameError Traceback (most recent call last)

<ipython-input-92-f356e0db002d> in <module>

1 del Language

----> 2 print(Language)

NameError: name 'Language' is not defined

In [93]:

```
print(Employee)
```

```
{'Name': 'Rohan', 'Age': 43, 'Dept': 'Account'}
```

In [94]:

```
Employee.pop("Age")
```

```
print(Employee)
```

```
{'Name': 'Rohan', 'Dept': 'Account'}
```

In [95]:

```
print(Employee)
```

```
{'Name': 'Rohan', 'Dept': 'Account'}
```

In [98]:

```
for key in Employee:
```

```
    print(key)
```

Name

Dept

In [99]:

```
for key in Employee:  
    print(Employee[key])
```

Rohan
Account

In [100]:

```
for value in Employee.values():  
    print(value)
```

Rohan
Account

In [101]:

```
for item in Employee.items():  
    print(item)
```

('Name', 'Rohan')
('Dept', 'Account')

In [102]:

```
print(Employee)
```

{'Name': 'Rohan', 'Dept': 'Account'}

In [103]:

```
Employee["Age"] = 56
```

In [104]:

```
print(Employee)
```

{'Name': 'Rohan', 'Dept': 'Account', 'Age': 56}

In [105]:

```
Employee["Dept"] = ("Account", "HR", "Marketing")
```

In [106]:

```
print(Employee)
```

{'Name': 'Rohan', 'Dept': ('Account', 'HR', 'Marketing'), 'Age': 56}

In [111]:

```
print(type(Employee))
```

<class 'dict'>

In [116]:

```
#Sample programs
# Dictionary  number : number * number    (1 to N)

N = int(input("Enter the number:: "))
reqdict1 = {}
for i in range(1,N+1):
    reqdict1[i] = i * i
print(reqdict1)
```

Enter the number:: 5

{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

In [130]:

```
#student record in dictionary
class Student:
    def getstudentdetails(obj):
        obj.__name = input("Enter the name::")
        obj.__rollno = int(input("Enter the roll no::"))
        obj.__branch = input("Enter the branch::")
        return obj.__rollno
    def displaystudentdetails(obj):
        print(obj.__name, obj.__rollno, obj.__branch)

studentrecord = {}
N = int(input("Number of students:: "))
for i in range(1,N+1):
    sobj = Student()
    rollno = sobj.getstudentdetails()
    studentrecord.setdefault(rollno,sobj)

for key in studentrecord:
    S = studentrecord.get(key,"Not available")
    if (isinstance(S,Student)):
        S.displaystudentdetails()
    else:
        print(S)
```

Number of students:: 3

Enter the name::Rohan

Enter the roll no::101

Enter the branch::CSE

Enter the name::Mohan

Enter the roll no::102

Enter the branch::CSE

Enter the name::Joe

Enter the roll no::103

Enter the branch::Mech

Rohan 101 CSE

Mohan 102 CSE

Joe 103 Mech

In [128]:

```
for key in studentrecord:  
    print(key)
```

102

101

In []: